The Relay — write-up

Code: `pipeline.py` `llm.py` `library.py` `standin.py` `run.py` · raw: `results.csv` (1,639 rows: 219 real, 1,420 stand-in), `answers.jsonl`, `calls.jsonl` · all tables: `python report.py`.

1 What I built and why

understand (gpt-oss-20b) → **retrieve** (BM25 tool) → **decide** → **draft** (gpt-oss-120b, streamed) → **check** (claim gate; one redraft). "Fast" = something useful within 6 s; I assume people leave at 8 s.

Shaping decision: each request carries a first-token deadline, and every step degrades rather than wait past it: 120b → 20b (separate limits) → cited quotes of the passages (0 tokens). The check runs *inside* the stream: a claim is released only if its cited ids were retrieved and its numbers appear in the cited passage. It costs nothing in time-to-first-token, but text arrives one claim at a time, so first token ≈ total.

2 The real service and my stand-in

Limit	Value I found	How I found it	Confidence
Requests per minute	30	429 body `RPM: Limit 30, Used 30`; at 1 req/s the **31st** got the first 429, then every other one	high
Tokens per minute	8,000	`x-ratelimit-limit-tokens`; 8 parallel ~2.7K-token prompts → 6 rejected, `TPM: Limit 8000, Used 5552`	high
Requests per day	1,000	`x-ratelimit-limit-requests`; reset 1m26.4s = 86,400/1,000, refills continuously; I used 210, never hit it	medium
Tokens per day	200,000	Groq docs only; never hit it	low
Requests in flight at once	no limit seen up to 40	40 simultaneous: 39 ok, 1 RPM 429	medium
What it returns when a limit is hit	HTTP 429, JSON naming limit + usage, `retry-after` 2–3 s	`probe_*` rows	high
Time until it recovers	under 1 s for a small request (retry-after said 3 s)	request 0.7 s after a TPM 429 succeeded	medium

Unexpected: 39 of 40 burst requests passed the 30/min limit (the `remaining` counter read 988 → 982 → 993: the counters agree only eventually). Groq also debits `max_tokens` at admission (79 + 200 → 279), so the free tier sustains only **~3 answers/min**.

Stand-in: same API, streaming, headers and 429 bodies; per-model windows with the up-front reservation; a daily cap, 5xx, slow successes, and slowdown as × (1 + in-flight/20). Calibrated on 61 real 120b calls: lognormal first token (median 0.52 s vs real 506 ms), 1,425 tokens/s, 1.6% slow (×5–15), 0 errors in 167 real calls. Load runs use 10× the free-tier limits. **Least like Groq:** its slowness comes only from my own load, never from other tenants, and it enforces limits exactly, so bursts never slip through.

3 Results

Table 2a — percentiles over requests that got a first token; `real_*` = Groq, others = stand-in.

Scenario	Real or stand-in	Requests sent	Duration (s)	Offered rate (per min)	First token p50 / p95 / p99 (ms)	Total time p50 / p95 / p99 (ms)
real_steady	real	66	1266	3	1141 / 2201 / 2703	1141 / 2205 / 2707
real_burst_cold_index	real	8	4	8 at once	3482 / 4013 / 4041	3492 / 4023 / 4051
real_burst	real	8	1	8 at once	920 / 1155 / 1211	925 / 1162 / 1219
steady	stand-in	71	181	24	1847 / 2503 / 2615	2247 / 2918 / 3037
burst	stand-in	100	8	100 at once	2065 / 4619 / 6948	2198 / 5047 / 7384
overload (sustained)	stand-in	376	185	125	2026 / 6264 / 7347	2401 / 6424 / 7769
overload_naive_retry	stand-in	376	187	125	2241 / 6971 / 7708	2622 / 7112 / 7832
overload_budget_retry	stand-in	376	184	125	2176 / 6401 / 7312	2514 / 6664 / 7621
halved_limit	stand-in	71	180	24	1830 / 2414 / 2817	2238 / 2836 / 3237
daily_exhausted	stand-in	50	109	27	5 / 1920 / 2059	5 / 2334 / 2466

Table 2b — cost at gpt-oss-120b $0.15/$0.60 and 20b $0.075/$0.30 per million tokens in/out. "Rate-limited" = answered with quoted passages instead of a model answer.

Scenario	Succeeded %	Rate-limited %	Timed out or given up %	Retries per request	Peak queue length	Tokens per answer	Cost per 1,000 answers
real_steady	98.5	0.0	1.5	0.00	0	1733	$0.30
real_burst_cold_index	50.0	0.0	50.0	0.00	7	1188	$0.21
real_burst	100.0	0.0	0.0	0.00	7	955	$0.19
steady	98.6	0.0	1.4	0.00	4	1050	$0.21
burst	100.0	0.0	0.0	0.00	99	587	$0.12
overload (sustained)	92.3	5.3	2.4	0.01	23	938	$0.15
overload_naive_retry	59.0	0.0	41.0	0.05	27	845	$0.17
overload_budget_retry	90.7	7.7	1.6	0.08	23	925	$0.15
halved_limit	100.0	0.0	0.0	0.00	3	1050	$0.21
daily_exhausted	30.0	70.0	0.0	0.00	3	1048	$0.21

`real_burst_cold_index` was my bug (the index was built lazily, 8 times in parallel, and the 2 s tool timeout fired); `real_burst` is the fixed rerun. The burst was 100% only because 91/100 ran in pressure mode with k=2 (weaker, still cited). Halving the limit cost nothing: the new TPM was learned from the headers.

Quality (22 questions I wrote and checked against the PDFs: 16 answerable, 6 not, e.g. "What GPUs trained H-Net?", where only Moshi names H100s): **13/16 right with citations, 1 wrong, 2 wrongly refused, 6/6 correctly refused**, at **1,019 tokens per answer**.

4 Where the time goes

`overload-…-0270` (stand-in), 10,577 ms: **waiting 0 ms**, **retrying 8,008 ms** (the 120b accepted the call, then sent no first token before my 8 s read timeout), **working 2,113 ms** (the 20b retry), 456 ms for retrieval and backoff. All of it came **before** the first token (10,200 ms). I know because `call()` charges each interval to `wait_ms`/`retry_ms`/`work_ms` on the row, and `retries=1, model=20b` agree. It also exposed a bug: the person left at 8.0 s, the same instant the timeout fired, and the cancel was swallowed (4 rows; now fixed and tested). On Groq, `real_steady-…-rerank-0018` shows the same shape: 7,691 ms of work and no first token by 8 s, well past the real 1.5 s p99.

5 When a limit is hit

Per-minute: a local per-model window mirrors Groq (prompt + `max_tokens` reserved; limits learned from headers; a 429 blocks until `retry-after`). Under overload a request is **delayed** only while it fits its deadline (mean 0.86 s), otherwise it **gets a weaker answer** from the 20b (128/376) or **cited**

quotes (5.3%). Nobody is turned away; the 2.4% "dropped" gave up on slow calls. Order is first-come. **Per-day:** a "per day" 429 or `remaining-requests: 0` marks the model exhausted, and everyone gets cited quotes in **5 ms (p50)**: weaker, never wrong, never waiting.

Retrieval: quality versus cost

Setting (piece size, pieces fetched, re-rank)	Questions asked	Correct and cited %	Wrong %	Correctly refused %	Tokens per answer	First token p50 / p95 (ms)	Total time p50 / p95 (ms)
120 words, 4, no	22	59.1	4.5	27.3	1019	1022 / 2507	1040 / 2512
250 words, 4, no	22	54.5	4.5	27.3	1776	1104 / 2016	1125 / 2016
120 words, 4, yes (20b picks 4 of 16)	22	54.5	0.0	27.3	2412	1459 / 2212	1460 / 2217

I chose 120 / 4 / no re-rank: the most correct at the fewest tokens. Bigger pieces cost 74% more tokens and were less accurate; re-ranking removed one wrong answer but wrongly refused two more, for +437 ms and 2.4× the tokens. On a TPM-bound tier, tokens per answer *is* throughput. **Near the limit:** no understand step, no re-rank, k=2 (quality in that mode not measured). (Correctly refused max = 27.3%.)

How this library will mislead a naive retriever

1. **It contradicts itself.** The English-to-French BLEU is 41.8 in the abstract and 41.0 in the page 8 text. 120-word pieces answered 41.8; 250-word pieces answered 41.0. Chunk size picked the "truth".
2. **Ablation numbers sit next to headline numbers.** "Moshi's practical latency?" retrieved an ablation on page 26 and answered **240 ms** (it is 200 ms, page 1). My gate passed it: it checks *support*, not *relevance*.
3. **Library-wide words act as stopwords.** "tokens", "layers" and "latency" are everywhere. "How many tokens was Helium pretrained on?" (2.1T, page 3) never made the top 4 and was wrongly refused in 3/3 settings.

6 Retries

Policy: a per-minute 429 waits out `retry-after` only within the deadline, else switches model; a per-day 429 never retries; a 5xx or a stall before the first token gets one retry (100–500 ms jitter) while retries stay under 10% of the last minute's calls; never after text was shown or the person left. **Made worse:** naive retries (3 tries, fixed 1 s, waiting for the window instead of switching), at the same load with 5% failures: **41.0% gave up vs 1.6%**, with fewer retries per request (0.05 vs 0.08). The damage was *waiting for capacity* while people's 8 s ran out.

7 What the person sees

At 3pm, within ~5 ms, the real reply (`daily_exhausted-…-0008`) to "What label smoothing value was used?":

> I can't write a full answer right now: today's model budget is used up (it refills gradually). The most relevant passages, quoted directly, are: - "… we employed label smoothing of value ϵls = 0.1 [36]…" (Attention Is All You Need, p.8) - "…Transformer (big) 28.4 41.8 2.3 · 1019…" (Attention Is All You Need, p.8)

It's true and cited and doesn't guess. The second quote is an unreadable table fragment, honest but weak. Capacity refills about one request every 86 s, so full answers return gradually.

8 What I'm unsure about, what I left out, what I learned

Least certain: that people give up at 8 s. If it's 3 s, my 6 s fallbacks fire too late and overload turns into "given up"; I'd fall back earlier and serve more weaker answers. **Not built:** an answer cache with request coalescing (bursts follow incidents, so people ask the same thing; left out so the tests measure the pipeline), and a model-based fact-checker (it would double calls on a TPM-bound tier; my gate misses pitfall 2). **Learned:** a "remaining" header is a hint, not a promise. Groq's counters agree only eventually (39/40 burst requests passed a 30/min limit; paced traffic failed on exactly the 31st), so a client limiter must be pessimistic and treat a 429 as a routing event, not an error.

Your feedback on Vaani

Stood out: a sharp focus (voice agents for Indian calling) with concrete claims, ~450 ms p90 latency and call screening, which is a real problem. "Teacher AI" showing prompt diffs before saving is the most thoughtful thing on the page. **Confused me:** "voice AI harness" and "dial only inside the window" are never explained, the page says both "Sub-500ms" and "~450 ms p90", and the stat counters first loaded as "0M+ / 0.0% / <0ms". Having just measured p50 vs p95 myself, I wanted to know which percentile and measured how. **One change:** put one measured number per claim, with how it was measured, next to a 20-second audio sample above the fold.

How I worked with the assistant

I used Claude Code. The full, unedited transcript is `transcript.txt` in this folder.

Prompts that mattered:

1. The brief + "Python, minimal code — what can be done in 5 lines shouldn't be done in 10 — follow skills.md, act like a senior engineer". This set the bar: the whole system, stand-in and runner included, is ~800 lines.
2. "Continue with the project", after I stopped its first command and installed a minimal-code plugin.
3. "Continue and fill everything", after a low-memory stop, once the real Groq runs were already safe in `results.csv`.

What it got wrong:

- Its citation checker silently turned correct answers into refusals, because gpt-oss writes 【C17】. Caught by reading one raw draft.
- Its grader failed 36 answers on spaces, and its first fix broke refusal grading. Caught by a test.
- It ran two load tests on one port. Caught from an impossible 1,959 s duration; the rows were quarantined and rerun.

Where I overruled it: I rejected its very first move, one big multi-purpose shell command, and put a minimal-code plugin in front of it before letting it continue. Later I overrode its default commit attribution so I'm the only contributor, with plain commit messages.

Distinctly mine:

- **The constraints:** minimal code, Python, and staying inside one free account, so I spent the scarce real budget on probes and quality runs and pushed load onto the stand-in.
- **The judgment calls:** the 8 s patience assumption; choosing 120/4/no re-rank on tokens-as-throughput grounds; keeping the bad cold-index and overlapping-run data visible instead of deleting it.